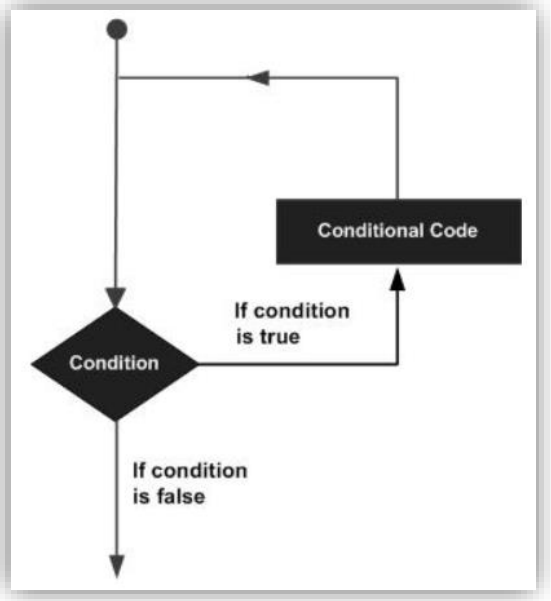
LECTURE – ITERATION
OOP (JAVA)

MR. PREM KUMAR
LECTURER

Dawood University of Engineering and Technology, Karachi

Loop Control

- Loops can execute a block of code as long as a specified condition is reached.
- Loops are handy because they save time, reduce errors, and they make code more readable.
- Java programming language provides the following types of loop to handle looping requirements.
  - **while loop**
  - **do/while loop**
  - **for loop**



while loop

- Repeats a statement or group of statements while a given condition is true. It tests the condition before executing the loop body.
- The while loop loops through a block of code as long as a specified condition is true:

Syntax

```
while (condition) {
  // code block to be executed
}
```

Example:

```
int i = 0;
while (i < 5) {
  System.out.println(i);
  i++;
}
```

```
public class Main {
  public static void main(String[] args) {
    int i = 0;
    while (i < 5) {
      System.out.println(i);
      i++;
    }
  }
}
```

Result:

```
0
1
2
3
4
```

do/while loop

- The do/while loop is a variant of the while loop.
- This loop will execute the code block once, before checking if the condition is true, then it will repeat the loop as long as the condition is true.

Syntax

```
do {
  // code block to be executed
}
while (condition);
```

Example:

```
int i = 0;
do {
  System.out.println(i);
  i++;
}
while (i < 5);
```

```
public class Main {
  public static void main(String[] args) {
    int i = 0;
    do {
      System.out.println(i);
      i++;
    }
    while (i < 5);
  }
}
```

Result:

```
0
1
2
3
4
```

for loop

- When you know exactly how many times you want to loop through a block of code, use the for loop instead of a while loop:

Syntax

```
for (statement 1; statement 2; statement 3) {
  // code block to be executed
}
```

- Statement 1 is executed (one time) before the execution of the code block.
- Statement 2 defines the condition for executing the code block.
- Statement 3 is executed (every time) after the code block has been executed.

for loop

- When you know exactly how many times you want to loop through a block of code, use the for loop instead of a while loop:

Example:

```
for (int i = 0; i < 5; i++) {
  System.out.println(i);
}
```

```
public class Main {
  public static void main(String[] args) {
    for (int i = 0; i < 5; i++) {
      System.out.println(i);
    }
  }
}
```

Result:

```
0
1
2
3
4
```

for loop

- When you know exactly how many times you want to loop through a block of code, use the for loop instead of a while loop:

Example:

```
for (int i = 0; i <= 10; i = i + 2) {
  System.out.println(i);
}
```

```
public class Main {
  public static void main(String[] args) {
    for (int i = 0; i <= 10; i = i + 2) {
      System.out.println(i);
    }
  }
}
```

Result:

```
0
2
4
6
8
10
```

for-each loop

- There is also a "**for-each**" loop, which is used exclusively to loop through elements in an **array**:

Syntax

```
for (type variableName : arrayName) {
  // code block to be executed
}
```

```
String[] cars = {"Volvo", "BMW", "Ford", "Mazda"};
for (String i : cars) {
  System.out.println(i);
}
```

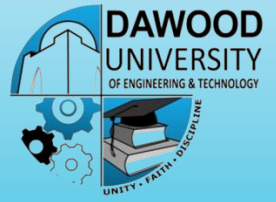
Example:

```
public class Main {
  public static void main(String[] args) {
    String[] cars = {"Volvo", "BMW", "Ford", "Mazda"};
    for (String i : cars) {
      System.out.println(i);
    }
  }
}
```

Result:

```
Volvo
BMW
Ford
Mazda
```

Break and Continue



Break

- The break statement can also be used to jump out of a **loop**.

Example:

```
for (int i = 0; i < 10; i++) {
  if (i == 4) {
    break;
  }
  System.out.println(i);
}
```

```
public class Main {
  public static void main(String[] args) {
    for (int i = 0; i < 10; i++) {
      if (i == 4) {
        break;
      }
      System.out.println(i);
    }
  }
}
```

Result:

```
0
1
2
3
```

Break and Continue

Continue

- The continue statement breaks one iteration (in the loop), if a specified condition occurs, and continues with the next iteration in the loop.

Example:

```
for (int i = 0; i < 10; i++) {
  if (i == 4) {
    continue;
  }
  System.out.println(i);
}
```

```
public class Main {
  public static void main(String[] args) {
    for (int i = 0; i < 10; i++) {
      if (i == 4) {
        continue;
      }
      System.out.println(i);
    }
  }
}
```

Result:

```
0
1
2
3
5
6
7
8
9
```

Break and Continue in while loop

- You can also use break and continue in while loops:

Break

Example:

```
public class Main {
  public static void main(String[] args) {
    int i = 0;
    while (i < 10) {
      System.out.println(i);
      i++;
      if (i == 4) {
        break;
      }
    }
  }
}
```

Result:

```
0
1
2
3
```

Break and Continue in while loop

- You can also use break and continue in while loops:

Continue

Example:

```
public class Main {
  public static void main(String[] args) {
    int i = 0;
    while (i < 10) {
      if (i == 4) {
        i++;
        continue;
      }
      System.out.println(i);
      i++;
    }
  }
}
```

Result:

```
0
1
2
3
5
6
7
8
9
```


THANK YOU!

Thanks ☺